Quantium Virtual Internship - Retail Strategy and Analytics - Task 1

In [165…
```
#Loading needed libraries
import pandas as pd
import numpy as np
import os
import warnings
warnings.filterwarnings('ignore')
import re
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import ttest_ind
```

In [2]:
```
print(os.getcwd())
```

```
/home/leda/Projects/Customer_segments
```

In [3]:
```
print(os.listdir())
```

```
['QVI_purchase_behaviour.csv', '.git', '.ipynb_checkpoints', 'README.md', 'QVI_transaction_data.xlsx', 'Retail_strategy.ipynb']
```

In [4]:
```
#Importing datasets
df_purchase = pd.read_csv("QVI_purchase_behaviour.csv")
df_transactions = pd.read_excel("QVI_transaction_data.xlsx")
```

Purchase dataset EDA

In [5]:
```
#Exploratory Data Analysis Purchase dataset
df_purchase.head()
```

Out[5]:

	LYLTY_CARD_NBR	LIFESTAGE	PREMIUM_CUSTOMER
0	1000	YOUNG SINGLES/COUPLES	Premium
1	1002	YOUNG SINGLES/COUPLES	Mainstream
2	1003	YOUNG FAMILIES	Budget
3	1004	OLDER SINGLES/COUPLES	Mainstream
4	1005	MIDAGE SINGLES/COUPLES	Mainstream

In [6]:
```
df_purchase.shape
```

Out[6]:
```
(72637, 3)
```

In [7]:
```
df_purchase.dtypes
```

Out[7]:
```
LYLTY_CARD_NBR       int64
LIFESTAGE           object
PREMIUM_CUSTOMER    object
dtype: object
```

In [8]:
```
df_purchase.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 72637 entries, 0 to 72636
Data columns (total 3 columns):
 #   Column            Non-Null Count  Dtype 
---  ------            --------------  ----- 
 0   LYLTY_CARD_NBR    72637 non-null  int64 
 1   LIFESTAGE         72637 non-null  object
 2   PREMIUM_CUSTOMER  72637 non-null  object
dtypes: int64(1), object(2)
memory usage: 1.7+ MB
```

In [9]:
```
df_purchase.duplicated().sum()
```

Out[9]:
```
0
```

Transaction dataset EDA

In [10]:
```
df_transactions.head()
```

Out[10]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES
0	43390	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	6.0
1	43599	1	1307	348	66	CCs Nacho Cheese 175g	3	6.3
2	43605	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	2.9
3	43329	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	5	15.0
4	43330	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	3	13.8

In [11]:
```
df_transactions.shape
```

Out[11]:
```
(264836, 8)
```

In [12]:
```
df_transactions.dtypes
```

Out[12]:
```
DATE                int64
STORE_NBR           int64
LYLTY_CARD_NBR      int64
TXN_ID              int64
PROD_NBR            int64
PROD_NAME          object
PROD_QTY            int64
TOT_SALES         float64
dtype: object
```

In [13]:
```
# Changing the format for the attribute "Date"
df_transactions["DATE"] = pd.to_datetime(df_transactions["DATE"], origin = "1899-12-30", unit = "D")
```

In [14]:
```
df_transactions.head()
```

Out[14]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES
0	2018-10-17	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	6.0
1	2019-05-14	1	1307	348	66	CCs Nacho Cheese 175g	3	6.3
2	2019-05-20	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	2.9
3	2018-08-17	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	5	15.0
4	2018-08-18	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	3	13.8

In [15]:
```
print(f'PROD_NAME: {df_transactions["PROD_NAME"].unique()}')
print(f'PROD_NAME: {df_transactions["PROD_NAME"].nunique()}')
```

```
PROD_NAME: ['Natural Chip        Compny SeaSalt175g' 'CCs Nacho Cheese    175g'
 'Smiths Crinkle Cut  Chips Chicken 170g'
 'Smiths Chip Thinly  S/Cream&Onion 175g'
 'Kettle Tortilla ChpsHny&Jlpno Chili 150g'
 'Old El Paso Salsa   Dip Tomato Mild 300g'
 'Smiths Crinkle Chips Salt & Vinegar 330g'
 'Grain Waves         Sweet Chilli 210g'
 'Doritos Corn Chip Mexican Jalapeno 150g'
 'Grain Waves Sour    Cream&Chives 210G'
 'Kettle Sensations   Siracha Lime 150g' 'Twisties Cheese     270g'
 'WW Crinkle Cut      Chicken 175g' 'Thins Chips Light&  Tangy 175g'
 'CCs Original 175g' 'Burger Rings 220g'
 'NCC Sour Cream &    Garden Chives 175g'
 'Doritos Corn Chip Southern Chicken 150g' 'Cheezels Cheese Box 125g'
 'Smiths Crinkle      Original 330g'
 'Infzns Crn Crnchers Tangy Gcamole 110g'
 'Kettle Sea Salt     And Vinegar 175g'
 'Smiths Chip Thinly  Cut Original 175g' 'Kettle Original 175g'
 'Red Rock Deli Thai  Chilli&Lime 150g' 'Pringles Sthrn FriedChicken 134g'
 'Pringles Sweet&Spcy BBQ 134g' 'Red Rock Deli SR    Salsa & Mzzrlla 150g'
 'Thins Chips         Originl saltd 175g'
 'Red Rock Deli Sp    Salt & Truffle 150G'
 'Smiths Thinly       Swt Chli&S/Cream175G' 'Kettle Chilli 175g'
 'Doritos Mexicana    170g' 'Smiths Crinkle Cut  French OnionDip 150g'
 'Natural ChipCo      Hony Soy Chckn175g'
 'Dorito Corn Chp     Supreme 380g' 'Twisties Chicken270g'
 'Smiths Thinly Cut   Roast Chicken 175g'
 'Smiths Crinkle Cut  Tomato Salsa 150g'
 'Kettle Mozzarella   Basil & Pesto 175g'
 'Infuzions Thai SweetChili PotatoMix 110g'
 'Kettle Sensations   Camembert & Fig 150g'
 'Smith Crinkle Cut   Mac N Cheese 150g'
 'Kettle Honey Soy    Chicken 175g' 'Thins Chips Seasonedchicken 175g'
 'Smiths Crinkle Cut  Salt & Vinegar 170g'
 'Infuzions BBQ Rib   Prawn Crackers 110g'
 'GrnWves Plus Btroot & Chilli Jam 180g'
 'Tyrrells Crisps     Lightly Salted 165g'
 'Kettle Sweet Chilli And Sour Cream 175g'
 'Doritos Salsa       Medium 300g' 'Kettle 135g Swt Pot Sea Salt'
 'Pringles SourCream  Onion 134g' 'Doritos Corn Chips  Original 170g'
 'Twisties Cheese     Burger 250g'
 'Old El Paso Salsa   Dip Chnky Tom Ht300g'
 'Cobs Popd Swt/Chlli &Sr/Cream Chips 110g'
 'Woolworths Mild     Salsa 300g'
 'Natural Chip Co     Tmato Hrb&Spce 175g'
 'Smiths Crinkle Cut  Chips Original 170g'
 'Cobs Popd Sea Salt  Chips 110g'
 'Smiths Crinkle Cut  Chips Chs&Onion170g'
 'French Fries Potato Chips 175g'
 'Old El Paso Salsa   Dip Tomato Med 300g'
 'Doritos Corn Chips  Cheese Supreme 170g'
 'Pringles Original   Crisps 134g' 'RRD Chilli&         Coconut 150g'
 'WW Original Corn    Chips 200g' 'Thins Potato Chips  Hot & Spicy 175g'
 'Cobs Popd Sour Crm  &Chives Chips 110g'
 'Smiths Crnkle Chip  Orgnl Big Bag 380g'
 'Doritos Corn Chips  Nacho Cheese 170g'
 'Kettle Sensations   BBQ&Maple 150g' 'WW D/Style Chip     Sea Salt 200g'
 'Pringles Chicken    Salt Crips 134g' 'WW Original Stacked Chips 160g'
 'Smiths Chip Thinly  CutSalt/Vinegr175g' 'Cheezels Cheese 330g'
 'Tostitos Lightly    Salted 175g' 'Thins Chips Salt &  Vinegar 175g'
 'Smiths Crinkle Cut  Chips Barbecue 170g' 'Cheetos Puffs 165g'
 'RRD Sweet Chilli &  Sour Cream 165g' 'WW Crinkle Cut      Original 175g'
 'Tostitos Splash Of  Lime 175g' 'Woolworths Medium   Salsa 300g'
 'Kettle Tortilla ChpsBtroot&Ricotta 150g' 'CCs Tasty Cheese    175g'
 'Woolworths Cheese   Rings 190g' 'Tostitos Smoked     Chipotle 175g'
 'Pringles Barbeque   134g' 'WW Supreme Cheese   Corn Chips 200g'
 'Pringles Mystery    Flavour 134g'
 'Tyrrells Crisps     Ched & Chives 165g'
 'Snbts Whlgrn Crisps Cheddr&Mstrd 90g' 'Cheetos Chs & Bacon Balls 190g'
 'Pringles Slt Vingar 134g' 'Infuzions SourCream&Herbs Veg Strws 110g'
 'Kettle Tortilla ChpsFeta&Garlic 150g'
 'Infuzions Mango     Chutny Papadums 70g'
 'RRD Steak &         Chimuchurri 150g' 'RRD Honey Soy       Chicken 165g'
 'Sunbites Whlegrn    Crisps Frch/Onin 90g' 'RRD Salt & Vinegar  165g'
 'Doritos Cheese      Supreme 330g' 'Smiths Crinkle Cut  Snag&Sauce 150g'
 'WW Sour Cream &OnionStacked Chips 160g' 'RRD Lime & Pepper   165g'
 'Natural ChipCo Sea  Salt & Vinegr 175g'
 'Red Rock Deli Chikn&Garlic Aioli 150g'
 'RRD SR Slow Rst     Pork Belly 150g' 'RRD Pc Sea Salt     165g'
```

```
 'Smith Crinkle Cut   Bolognese 150g' 'Doritos Salsa Mild  300g']
PROD_NAME: 114
```

In [16]:
```
df_transactions["PROD_NAME"] = df_transactions["PROD_NAME"].str.lower()
```

In [17]:
```
print(f'PROD_NAME: {df_transactions["PROD_NAME"].unique()}')
print(f'PROD_NAME: {df_transactions["PROD_NAME"].nunique()}')
```

```
PROD_NAME: ['natural chip        compny seasalt175g' 'ccs nacho cheese    175g'
 'smiths crinkle cut  chips chicken 170g'
 'smiths chip thinly  s/cream&onion 175g'
 'kettle tortilla chpshny&jlpno chili 150g'
 'old el paso salsa   dip tomato mild 300g'
 'smiths crinkle chips salt & vinegar 330g'
 'grain waves         sweet chilli 210g'
 'doritos corn chip mexican jalapeno 150g'
 'grain waves sour    cream&chives 210g'
 'kettle sensations   siracha lime 150g' 'twisties cheese     270g'
 'ww crinkle cut      chicken 175g' 'thins chips light&  tangy 175g'
 'ccs original 175g' 'burger rings 220g'
 'ncc sour cream &    garden chives 175g'
 'doritos corn chip southern chicken 150g' 'cheezels cheese box 125g'
 'smiths crinkle      original 330g'
 'infzns crn crnchers tangy gcamole 110g'
 'kettle sea salt     and vinegar 175g'
 'smiths chip thinly  cut original 175g' 'kettle original 175g'
 'red rock deli thai  chilli&lime 150g' 'pringles sthrn friedchicken 134g'
 'pringles sweet&spcy bbq 134g' 'red rock deli sr    salsa & mzzrlla 150g'
 'thins chips         originl saltd 175g'
 'red rock deli sp    salt & truffle 150g'
 'smiths thinly       swt chli&s/cream175g' 'kettle chilli 175g'
 'doritos mexicana    170g' 'smiths crinkle cut  french oniondip 150g'
 'natural chipco      hony soy chckn175g'
 'dorito corn chp     supreme 380g' 'twisties chicken270g'
 'smiths thinly cut   roast chicken 175g'
 'smiths crinkle cut  tomato salsa 150g'
 'kettle mozzarella   basil & pesto 175g'
 'infuzions thai sweetchili potatomix 110g'
 'kettle sensations   camembert & fig 150g'
 'smith crinkle cut   mac n cheese 150g'
 'kettle honey soy    chicken 175g' 'thins chips seasonedchicken 175g'
 'smiths crinkle cut  salt & vinegar 170g'
 'infuzions bbq rib   prawn crackers 110g'
 'grnwves plus btroot & chilli jam 180g'
 'tyrrells crisps     lightly salted 165g'
 'kettle sweet chilli and sour cream 175g'
 'doritos salsa       medium 300g' 'kettle 135g swt pot sea salt'
 'pringles sourcream  onion 134g' 'doritos corn chips  original 170g'
 'twisties cheese     burger 250g'
 'old el paso salsa   dip chnky tom ht300g'
 'cobs popd swt/chlli &sr/cream chips 110g'
 'woolworths mild     salsa 300g'
 'natural chip co     tmato hrb&spce 175g'
 'smiths crinkle cut  chips original 170g'
 'cobs popd sea salt  chips 110g'
 'smiths crinkle cut  chips chs&onion170g'
 'french fries potato chips 175g'
 'old el paso salsa   dip tomato med 300g'
 'doritos corn chips  cheese supreme 170g'
 'pringles original   crisps 134g' 'rrd chilli&         coconut 150g'
 'ww original corn    chips 200g' 'thins potato chips  hot & spicy 175g'
 'cobs popd sour crm  &chives chips 110g'
 'smiths crnkle chip  orgnl big bag 380g'
 'doritos corn chips  nacho cheese 170g'
 'kettle sensations   bbq&maple 150g' 'ww d/style chip     sea salt 200g'
 'pringles chicken    salt crips 134g' 'ww original stacked chips 160g'
 'smiths chip thinly  cutsalt/vinegr175g' 'cheezels cheese 330g'
 'tostitos lightly    salted 175g' 'thins chips salt &  vinegar 175g'
 'smiths crinkle cut  chips barbecue 170g' 'cheetos puffs 165g'
 'rrd sweet chilli &  sour cream 165g' 'ww crinkle cut      original 175g'
 'tostitos splash of  lime 175g' 'woolworths medium   salsa 300g'
 'kettle tortilla chpsbtroot&ricotta 150g' 'ccs tasty cheese    175g'
 'woolworths cheese   rings 190g' 'tostitos smoked     chipotle 175g'
 'pringles barbeque   134g' 'ww supreme cheese   corn chips 200g'
 'pringles mystery    flavour 134g'
 'tyrrells crisps     ched & chives 165g'
 'snbts whlgrn crisps cheddr&mstrd 90g' 'cheetos chs & bacon balls 190g'
 'pringles slt vingar 134g' 'infuzions sourcream&herbs veg strws 110g'
 'kettle tortilla chpsfeta&garlic 150g'
 'infuzions mango     chutny papadums 70g'
 'rrd steak &         chimuchurri 150g' 'rrd honey soy       chicken 165g'
 'sunbites whlegrn    crisps frch/onin 90g' 'rrd salt & vinegar  165g'
 'doritos cheese      supreme 330g' 'smiths crinkle cut  snag&sauce 150g'
 'ww sour cream &onionstacked chips 160g' 'rrd lime & pepper   165g'
 'natural chipco sea  salt & vinegr 175g'
 'red rock deli chikn&garlic aioli 150g'
 'rrd sr slow rst     pork belly 150g' 'rrd pc sea salt     165g'
```

```
 'smith crinkle cut   bolognese 150g' 'doritos salsa mild  300g']
PROD_NAME: 114
```

In [18]:
```
pattern = r"\b(chip|chips|crips|crips|potato|potatoes)\b"

mask_has_chip = df_transactions["PROD_NAME"].str.contains(
    pattern,
    case = False,
    na = False,
    regex = True
)

df_no_chips = df_transactions[~mask_has_chip]

print(df_no_chips[["PROD_NAME"]])
print(f'Total rows without chip-related words: {len(df_no_chips)}')
```

```
                                     PROD_NAME
1                       ccs nacho cheese    175g
4       kettle tortilla chpshny&jlpno chili 150g
5       old el paso salsa   dip tomato mild 300g
7          grain waves         sweet chilli 210g
9          grain waves sour    cream&chives 210g
...                                       ...
264829  infuzions sourcream&herbs veg strws 110g
264831   kettle sweet chilli and sour cream 175g
264832            tostitos splash of  lime 175g
264833                 doritos mexicana    170g
264835            tostitos splash of  lime 175g

[193317 rows x 1 columns]
Total rows without chip-related words: 193317
```

In [19]:
```
print(f'PROD_NAME: {df_no_chips["PROD_NAME"].unique()}')
print(f'PROD_NAME: {df_no_chips["PROD_NAME"].nunique()}')
```

```
PROD_NAME: ['ccs nacho cheese    175g' 'kettle tortilla chpshny&jlpno chili 150g'
 'old el paso salsa   dip tomato mild 300g'
 'grain waves         sweet chilli 210g'
 'grain waves sour    cream&chives 210g'
 'kettle sensations   siracha lime 150g' 'twisties cheese     270g'
 'ww crinkle cut      chicken 175g' 'ccs original 175g'
 'burger rings 220g' 'ncc sour cream &    garden chives 175g'
 'cheezels cheese box 125g' 'smiths crinkle      original 330g'
 'infzns crn crnchers tangy gcamole 110g'
 'kettle sea salt     and vinegar 175g' 'kettle original 175g'
 'red rock deli thai  chilli&lime 150g' 'pringles sthrn friedchicken 134g'
 'pringles sweet&spcy bbq 134g' 'red rock deli sr    salsa & mzzrlla 150g'
 'red rock deli sp    salt & truffle 150g'
 'smiths thinly       swt chli&s/cream175g' 'kettle chilli 175g'
 'doritos mexicana    170g' 'smiths crinkle cut  french oniondip 150g'
 'natural chipco      hony soy chckn175g'
 'dorito corn chp     supreme 380g' 'twisties chicken270g'
 'smiths thinly cut   roast chicken 175g'
 'smiths crinkle cut  tomato salsa 150g'
 'kettle mozzarella   basil & pesto 175g'
 'infuzions thai sweetchili potatomix 110g'
 'kettle sensations   camembert & fig 150g'
 'smith crinkle cut   mac n cheese 150g'
 'kettle honey soy    chicken 175g'
 'smiths crinkle cut  salt & vinegar 170g'
 'infuzions bbq rib   prawn crackers 110g'
 'grnwves plus btroot & chilli jam 180g'
 'tyrrells crisps     lightly salted 165g'
 'kettle sweet chilli and sour cream 175g'
 'doritos salsa       medium 300g' 'kettle 135g swt pot sea salt'
 'pringles sourcream  onion 134g' 'twisties cheese     burger 250g'
 'old el paso salsa   dip chnky tom ht300g'
 'woolworths mild     salsa 300g'
 'old el paso salsa   dip tomato med 300g'
 'pringles original   crisps 134g' 'rrd chilli&         coconut 150g'
 'kettle sensations   bbq&maple 150g' 'cheezels cheese 330g'
 'tostitos lightly    salted 175g' 'cheetos puffs 165g'
 'rrd sweet chilli &  sour cream 165g' 'ww crinkle cut      original 175g'
 'tostitos splash of  lime 175g' 'woolworths medium   salsa 300g'
 'kettle tortilla chpsbtroot&ricotta 150g' 'ccs tasty cheese    175g'
 'woolworths cheese   rings 190g' 'tostitos smoked     chipotle 175g'
 'pringles barbeque   134g' 'pringles mystery    flavour 134g'
 'tyrrells crisps     ched & chives 165g'
 'snbts whlgrn crisps cheddr&mstrd 90g' 'cheetos chs & bacon balls 190g'
 'pringles slt vingar 134g' 'infuzions sourcream&herbs veg strws 110g'
 'kettle tortilla chpsfeta&garlic 150g'
 'infuzions mango     chutny papadums 70g'
 'rrd steak &         chimuchurri 150g' 'rrd honey soy       chicken 165g'
 'sunbites whlegrn    crisps frch/onin 90g' 'rrd salt & vinegar  165g'
 'doritos cheese      supreme 330g' 'smiths crinkle cut  snag&sauce 150g'
 'rrd lime & pepper   165g' 'natural chipco sea  salt & vinegr 175g'
 'red rock deli chikn&garlic aioli 150g'
 'rrd sr slow rst     pork belly 150g' 'rrd pc sea salt     165g'
 'smith crinkle cut   bolognese 150g' 'doritos salsa mild  300g']
PROD_NAME: 83
```

In [20]:

```python
chip_brands = [
    "smiths", "kettle", "doritos", "twisties", "pringles",
    "tyrrells", "cheezels", "tostitos", "snbts", "rrd", "natural chipco", "ccs"
]

pattern = r"\b(chip|chips|crip|crips|potato|potatoes)\b"

df_transactions["is_chip"] = (
    df_transactions["PROD_NAME"].str.contains(pattern, case=False, na=False, regex=True) |
    df_transactions["PROD_NAME"].str.lower().str.contains("|".join([b.lower() for b in chip_brands]))
)

df_no_chips = df_transactions[~df_transactions["is_chip"]]

print(df_no_chips[["PROD_NAME"]].head(20))
print(f"Total non-chip rows: {len(df_no_chips)}")
```

```
                                PROD_NAME
5   old el paso salsa   dip tomato mild 300g
7      grain waves         sweet chilli 210g
9      grain waves sour    cream&chives 210g
13          ww crinkle cut      chicken 175g
16                        burger rings 220g
17    ncc sour cream &    garden chives 175g
21    ncc sour cream &    garden chives 175g
22    infzns crn crnchers tangy gcamole 110g
25  old el paso salsa   dip tomato mild 300g
28      red rock deli thai  chilli&lime 150g
29    infzns crn crnchers tangy gcamole 110g
32  red rock deli sr    salsa & mzzrlla 150g
34   red rock deli sp    salt & truffle 150g
41          dorito corn chp     supreme 380g
46    infzns crn crnchers tangy gcamole 110g
47  infuzions thai sweetchili potatomix 110g
50     smith crinkle cut   mac n cheese 150g
55   infuzions bbq rib   prawn crackers 110g
56     grnwves plus btroot & chilli jam 180g
64  infuzions thai sweetchili potatomix 110g
Total non-chip rows: 57954
```

In [21]:

```python
# Likely chip keywords
chip_keywords = [
    "chip","chips","crisp","crisps","potato","potatoes",
    "crinkle cut","tortilla","sensations","original"
]

# Known chip brands
chip_brands = [
    "smiths", "kettle", "doritos", "twisties", "pringles",
    "tyrrells", "cheezels", "tostitos", "snbts", "rrd", "natural chipco", "ccs",
]

# Obvious non-chip categories
non_chip_keywords = [
    "salsa","dip","prawn crackers","nuts","papadums","sweet","chocolate","cheese sticks"
]

# Lowercase product names for consistent matching
prod_lower = df_transactions["PROD_NAME"].str.lower()

# Step 1: Identify likely chips
is_chip = prod_lower.str.contains("|".join(chip_keywords), na=False) | \
          prod_lower.str.contains("|".join([b.lower() for b in chip_brands]), na=False)

# Step 2: Identify obvious non-chips
is_non_chip = prod_lower.str.contains("|".join(non_chip_keywords), na=False)

# Step 3: Final flag
df_transactions["is_chip_final"] = is_chip & (~is_non_chip)

# Rows that are definitely not chips
df_no_chips = df_transactions[~df_transactions["is_chip_final"]]

print(df_no_chips["PROD_NAME"].head(20))
print(f"Total non-chip rows: {len(df_no_chips)}")
```

```
5     old el paso salsa   dip tomato mild 300g
7        grain waves         sweet chilli 210g
9        grain waves sour    cream&chives 210g
16                           burger rings 220g
17      ncc sour cream &    garden chives 175g
21      ncc sour cream &    garden chives 175g
22      infzns crn crnchers tangy gcamole 110g
25    old el paso salsa   dip tomato mild 300g
28        red rock deli thai  chilli&lime 150g
29      infzns crn crnchers tangy gcamole 110g
31                pringles sweet&spcy bbq 134g
32    red rock deli sr    salsa & mzzrlla 150g
34     red rock deli sp    salt & truffle 150g
39    smiths crinkle cut  french oniondip 150g
41            dorito corn chp     supreme 380g
44       smiths crinkle cut  tomato salsa 150g
46      infzns crn crnchers tangy gcamole 110g
47    infuzions thai sweetchili potatomix 110g
55     infuzions bbq rib   prawn crackers 110g
56       grnwves plus btroot & chilli jam 180g
Name: PROD_NAME, dtype: object
Total non-chip rows: 64404
```

In [22]:
```python
print(f'PROD_NAME: {df_no_chips["PROD_NAME"].unique()}')
print(f'PROD_NAME: {df_no_chips["PROD_NAME"].nunique()}')
```

```
PROD_NAME: ['old el paso salsa   dip tomato mild 300g'
 'grain waves         sweet chilli 210g'
 'grain waves sour    cream&chives 210g' 'burger rings 220g'
 'ncc sour cream &    garden chives 175g'
 'infzns crn crnchers tangy gcamole 110g'
 'red rock deli thai  chilli&lime 150g' 'pringles sweet&spcy bbq 134g'
 'red rock deli sr    salsa & mzzrlla 150g'
 'red rock deli sp    salt & truffle 150g'
 'smiths crinkle cut  french oniondip 150g'
 'dorito corn chp     supreme 380g'
 'smiths crinkle cut  tomato salsa 150g'
 'infuzions thai sweetchili potatomix 110g'
 'infuzions bbq rib   prawn crackers 110g'
 'grnwves plus btroot & chilli jam 180g'
 'kettle sweet chilli and sour cream 175g'
 'doritos salsa       medium 300g'
 'old el paso salsa   dip chnky tom ht300g'
 'woolworths mild     salsa 300g'
 'old el paso salsa   dip tomato med 300g' 'cheetos puffs 165g'
 'rrd sweet chilli &  sour cream 165g' 'woolworths medium   salsa 300g'
 'woolworths cheese   rings 190g' 'cheetos chs & bacon balls 190g'
 'infuzions sourcream&herbs veg strws 110g'
 'infuzions mango     chutny papadums 70g'
 'red rock deli chikn&garlic aioli 150g' 'doritos salsa mild  300g']
PROD_NAME: 30
```

Notes: This could be taken as products that don't represents chips and therefore we could proceed to set them apart in order to get the dataset clean. However, as a result of further investigation it has been found that some of this are other chip brands, they will be included in the brands keywords.

In [23]:
```python
#Chip keywords
chip_keywords = [
    "chip","chips","crisp","crisps","potato","potatoes",
    "crinkle cut","tortilla","sensations","original","rings","potatomix"
]

#Chip brands
chip_brands = [
    "smiths", "kettle", "doritos", "twisties", "pringles",
    "tyrrells", "cheezels", "tostitos", "snbts", "rrd", "natural chipco", "ccs",
    "grain waves", "ncc", "red rock deli", "dorito", "infuzions", "rrd", "infzns",
    "grnwves", "cheetos"
]

#Non-chip categories
non_chip_keywords = [
    "salsa","dip","nuts","papadums","chocolate","cheese sticks"
]

#Lowercase product names
prod_lower = df_transactions["PROD_NAME"].str.lower()

#Identify likely chips
is_chip = prod_lower.str.contains("|".join(chip_keywords), na=False) | \
```

```
          prod_lower.str.contains("|".join([b.lower() for b in chip_brands]), na=False)

# Step 2: Identify obvious non-chips
is_non_chip = prod_lower.str.contains("|".join(non_chip_keywords), na=False)

# Step 3: Final flag
df_transactions["is_chip_final"] = is_chip & (~is_non_chip)

# Rows that are definitely not chips
df_no_chips = df_transactions[~df_transactions["is_chip_final"]]

print(df_no_chips["PROD_NAME"].head(20))
print(f"Total non-chip rows: {len(df_no_chips)}")
```

```
5      old el paso salsa   dip tomato mild 300g
25     old el paso salsa   dip tomato mild 300g
32     red rock deli sr    salsa & mzzrlla 150g
39     smiths crinkle cut  french oniondip 150g
44        smiths crinkle cut  tomato salsa 150g
63              doritos salsa       medium 300g
71     old el paso salsa   dip chnky tom ht300g
74               woolworths mild     salsa 300g
83     smiths crinkle cut  french oniondip 150g
87      old el paso salsa   dip tomato med 300g
114     old el paso salsa   dip tomato med 300g
131     old el paso salsa   dip tomato med 300g
149    smiths crinkle cut  french oniondip 150g
158              woolworths medium   salsa 300g
178    old el paso salsa   dip chnky tom ht300g
189    red rock deli sr    salsa & mzzrlla 150g
190       smiths crinkle cut  tomato salsa 150g
207    old el paso salsa   dip tomato mild 300g
220    smiths crinkle cut  french oniondip 150g
222    red rock deli sr    salsa & mzzrlla 150g
Name: PROD_NAME, dtype: object
Total non-chip rows: 21039
```

In [24]:

```
df_transactions = df_transactions[df_transactions["is_chip_final"]]
df_transactions.head()
df_transactions.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 243797 entries, 0 to 264835
Data columns (total 10 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   DATE            243797 non-null  datetime64[ns]
 1   STORE_NBR       243797 non-null  int64
 2   LYLTY_CARD_NBR  243797 non-null  int64
 3   TXN_ID          243797 non-null  int64
 4   PROD_NBR        243797 non-null  int64
 5   PROD_NAME       243797 non-null  object
 6   PROD_QTY        243797 non-null  int64
 7   TOT_SALES       243797 non-null  float64
 8   is_chip         243797 non-null  bool
 9   is_chip_final   243797 non-null  bool
dtypes: bool(2), datetime64[ns](1), float64(1), int64(5), object(1)
memory usage: 17.2+ MB
```

In [25]:

```
df_transactions.drop(columns=["is_chip","is_chip_final"], inplace = True)
```

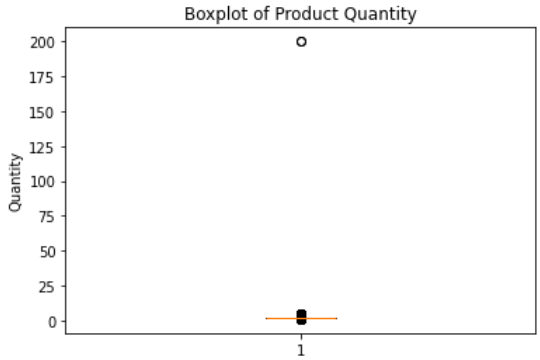
In [26]:

```
df_transactions.describe()
```

Out[26]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_QTY	TOT_SALES
count	243797	243797.000000	2.437970e+05	2.437970e+05	243797.000000	243797.000000	243797.000000
mean	2018-12-30 01:17:43.105780736	135.050960	1.355349e+05	1.351317e+05	56.166487	1.908284	7.352755
min	2018-07-01 00:00:00	1.000000	1.000000e+03	1.000000e+00	1.000000	1.000000	1.700000
25%	2018-09-30 00:00:00	70.000000	7.001000e+04	6.753800e+04	26.000000	2.000000	5.800000
50%	2018-12-30 00:00:00	130.000000	1.303720e+05	1.352060e+05	53.000000	2.000000	7.400000
75%	2019-03-31 00:00:00	203.000000	2.030860e+05	2.026630e+05	86.000000	2.000000	8.800000
max	2019-06-30 00:00:00	272.000000	2.373711e+06	2.415841e+06	114.000000	200.000000	650.000000
std	NaN	76.802871	8.077656e+04	7.816603e+04	33.642980	0.662536	3.081209

In [27]:
```
plt.boxplot(df_transactions["PROD_QTY"])
plt.title("Boxplot of Product Quantity")
plt.ylabel("Quantity")
plt.show()
```



In [28]:
```
count_200 = (df_transactions["PROD_QTY"] == 200).sum()
print(f'Number of transactions with quantity 200: {count_200}')
```

```
Number of transactions with quantity 200: 2
```

In [29]:
```
df_transactions[df_transactions["PROD_QTY"] == 200]
```

Out[29]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES
69762	2018-08-19	226	226000	226201	4	dorito corn chp supreme 380g	200	650.0
69763	2019-05-20	226	226000	226210	4	dorito corn chp supreme 380g	200	650.0

In [30]:
```
df_transactions["PROD_QTY"].value_counts()
```

Out[30]:
```
PROD_QTY
2      217505
1       25114
5         411
3         405
4         360
200         2
Name: count, dtype: int64
```

The sale for 200 was only twice and by the same customer, a quick check if the customer has made other purchases or is not a regular customer, to make decisions about this data.

In [31]:
```
df_transactions[df_transactions["LYLTY_CARD_NBR"] == 226000]
```

Out[31]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES
69762	2018-08-19	226	226000	226201	4	dorito corn chp supreme 380g	200	650.0
69763	2019-05-20	226	226000	226210	4	dorito corn chp supreme 380g	200	650.0

The customer has just made those two big purchases, he is not a regular customer. He'll be removed from the dataset his purchase is way out of range compare to the other ones.

In [32]:
```
df_transactions = df_transactions.drop(
    df_transactions[df_transactions["LYLTY_CARD_NBR"] == 226000].index
)
```

In [33]:
```
df_transactions[df_transactions["LYLTY_CARD_NBR"] == 226000]
```

Out[33]:

DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES

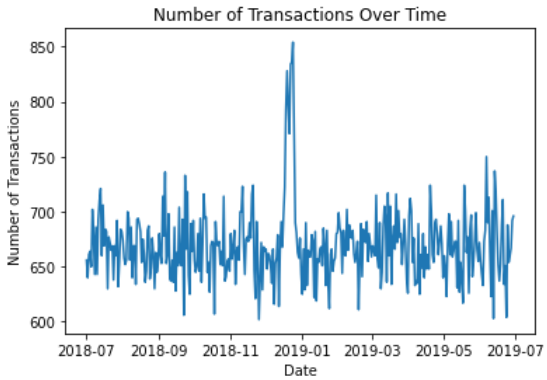
In [50]:
```
df_transactions_per_day = df_transactions.groupby("DATE").size().reset_index(name="count")
```

In [51]:
```
df_transactions_per_day.head()
```

Out[51]:

	DATE	count
0	2018-07-01	656
1	2018-07-02	640
2	2018-07-03	661
3	2018-07-04	664
4	2018-07-05	650

In [56]:
```
plt.figsize=(12,5))
plt.plot(df_transactions_per_day["DATE"],df_transactions_per_day["count"])
plt.title("Number of Transactions Over Time")
plt.xlabel("Date")
plt.ylabel("Number of Transactions")
plt.show()
```



In [58]:
```
df_transactions_per_day.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 364 entries, 0 to 363
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   DATE    364 non-null    datetime64[ns]
 1   count   364 non-null    int64
dtypes: datetime64[ns](1), int64(1)
memory usage: 5.8 KB
```

There's only 364 dates which indicates a missing date. Date go from 1 Jul 2018 to 30 Jun 2019. Fill in the missing day.

In [59]:
```
date_spine = pd.DataFrame({
    "DATE": pd.date_range(start="2018-07-01", end="2019-06-30")
})

transactions_per_day = df_transactions.groupby("DATE").size().reset_index(name = "TRANSACTIONS")

df_transactions_final = date_spine.merge(transactions_per_day, on = "DATE", how = "left")

df_transactions_final["TRANSACTIONS"] = df_transactions_final["TRANSACTIONS"].fillna(0)

print(df_transactions_final)
```

```
          DATE  TRANSACTIONS
0   2018-07-01         656.0
1   2018-07-02         640.0
2   2018-07-03         661.0
3   2018-07-04         664.0
4   2018-07-05         650.0
..         ...           ...
360 2019-06-26         654.0
361 2019-06-27         660.0
362 2019-06-28         667.0
363 2019-06-29         692.0
364 2019-06-30         696.0

[365 rows x 2 columns]
```

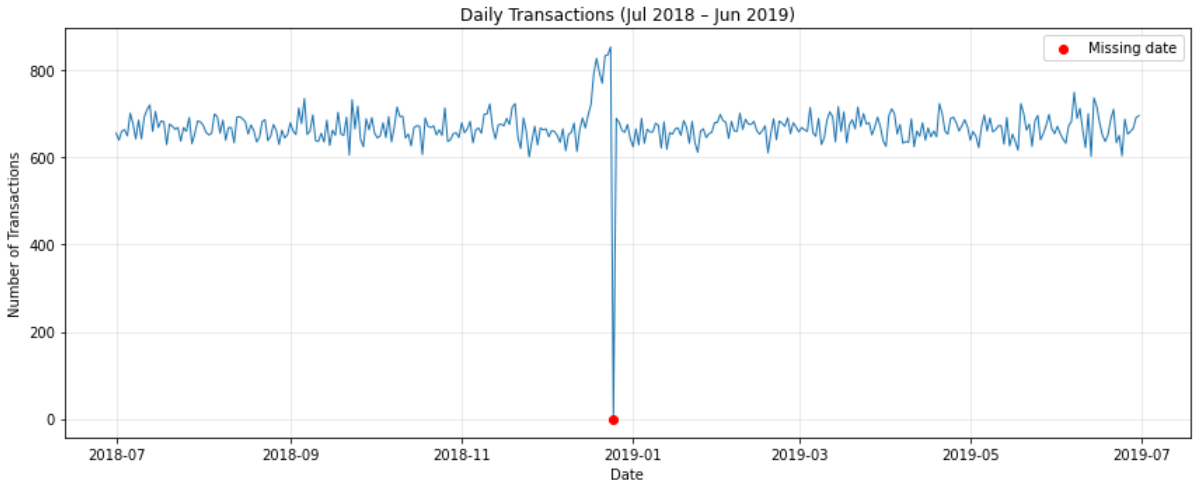
In [60]:
```
plt.figure(figsize=(12,5))
plt.plot(df_transactions_final['DATE'], df_transactions_final['TRANSACTIONS'], linewidth=1)

# Highlight missing day (where transactions == 0)
missing = df_transactions_final[df_transactions_final['TRANSACTIONS'] == 0]
```

```
plt.scatter(missing['DATE'], missing['TRANSACTIONS'], color='red', label='Missing date', zorder=5)

plt.title('Daily Transactions (Jul 2018 – Jun 2019)')
plt.xlabel('Date')
plt.ylabel('Number of Transactions')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



There is an increase of purchases in December and a break at the end of the month.

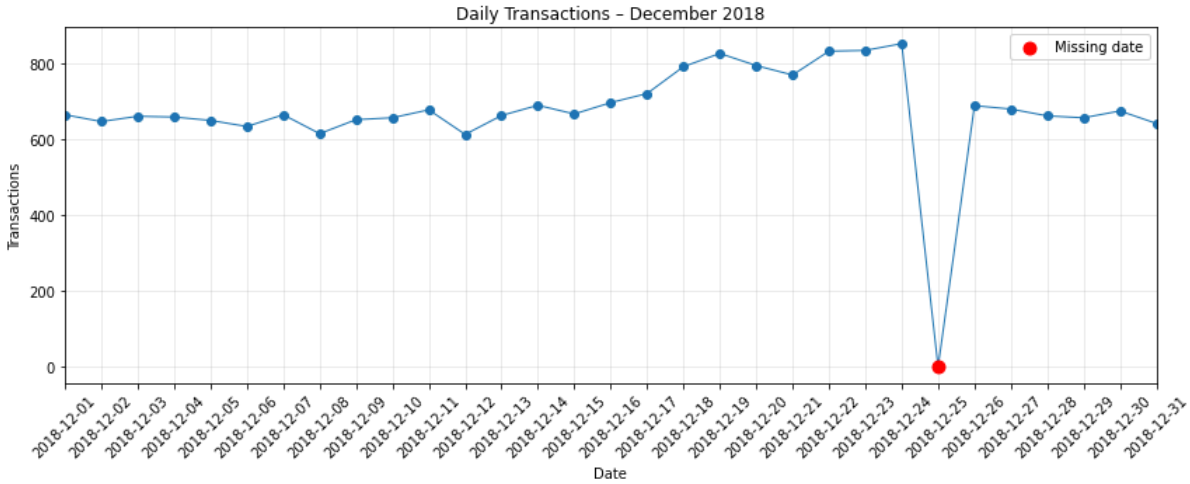
In [61]:
```
dec_df = df_transactions_final[
    (df_transactions_final['DATE'] >= '2018-12-01') &
    (df_transactions_final['DATE'] <= '2018-12-31')
].copy()

plt.figure(figsize=(12,5))
plt.plot(df_transactions_final['DATE'], df_transactions_final['TRANSACTIONS'], marker='o', linewidth=1)
plt.xlim(pd.Timestamp('2018-12-01'), pd.Timestamp('2018-12-31'))

missing = dec_df[dec_df['TRANSACTIONS'] == 0]
plt.scatter(missing['DATE'], missing['TRANSACTIONS'],
            color='red', s=80, label='Missing date', zorder=5)

plt.title('Daily Transactions – December 2018')
plt.xlabel('Date')
plt.ylabel('Transactions')
plt.xticks(dec_df['DATE'],rotation=45)
plt.grid(alpha=0.3)
plt.legend()

plt.tight_layout()
plt.show()
```



The increase in sales occurs leading up to Christmas and it's zero sales on Christmas day. This day the shop was closed for Christmas day. Tha data has no longer outlies.

In [63]:
```
df_transactions.head()
```

Out[63]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES	transactions_per_day
0	2018-10-17	1	1000	1	5	natural chip compny seasalt175g	2	6.0	NaN
1	2019-05-14	1	1307	348	66	ccs nacho cheese 175g	3	6.3	NaN
2	2019-05-20	1	1343	383	61	smiths crinkle cut chips chicken 170g	2	2.9	NaN
3	2018-08-17	2	2373	974	69	smiths chip thinly s/cream&onion 175g	5	15.0	NaN
4	2018-08-18	2	2426	1038	108	kettle tortilla chpshny&jlpno chili 150g	3	13.8	NaN

The data has no outliers so now new features such as Brand of chips and package size will be created

Creating new useful attributes

In [64]:
```
#Creating Brand attribute
df_transactions["BRAND"] = df_transactions["PROD_NAME"].str.split().str[0]
df_transactions.head()
```

Out[64]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES	transactions_per_day	BRAND
0	2018-10-17	1	1000	1	5	natural chip compny seasalt175g	2	6.0	NaN	natural
1	2019-05-14	1	1307	348	66	ccs nacho cheese 175g	3	6.3	NaN	ccs
2	2019-05-20	1	1343	383	61	smiths crinkle cut chips chicken 170g	2	2.9	NaN	smiths
3	2018-08-17	2	2373	974	69	smiths chip thinly s/cream&onion 175g	5	15.0	NaN	smiths
4	2018-08-18	2	2426	1038	108	kettle tortilla chpshny&jlpno chili 150g	3	13.8	NaN	kettle

In [65]:
```
print(f'BRAND: {df_transactions["BRAND"].unique()}')
print(f'BRAND: {df_transactions["BRAND"].nunique()}')
```

```
BRAND: ['natural' 'ccs' 'smiths' 'kettle' 'grain' 'doritos' 'twisties' 'ww'
 'thins' 'burger' 'ncc' 'cheezels' 'infzns' 'red' 'pringles' 'dorito'
 'infuzions' 'smith' 'grnwves' 'tyrrells' 'cobs' 'french' 'rrd' 'tostitos'
 'cheetos' 'woolworths' 'snbts' 'sunbites']
BRAND: 28
```

In [85]:
```
brand_map = {
    'ncc': 'Natural chip co.', 'natural': 'Natural chip co.',
    'smith': 'Smiths', 'smiths': 'Smiths',
    'dorito': 'Doritos','doritos': 'Doritos',
    'infzns': 'Infuzions', 'infuzions': 'Infuzions',
    'rrd': 'Red Rock Deli', 'red': 'Red Rock Deli',
    'grnwves': 'Grain Waves', 'grain': 'Grain Waves',
    'snbts': 'Sunbites', 'sunbites': 'Sunbites',
    'ww': 'Woolworths', 'woolworths': 'Woolworths',
    'ccs': 'CCs', 'kettle': 'Kettle',
    'thins': 'Thins', 'cheezels': 'Cheezels',
    'tostitos': 'Tostitos', 'cheetos': 'Cheetos',
    'twisties': 'Twisties', 'pringles': 'Pringles',
    'tyrrells': 'Tyrrells', 'cobs': 'Cobs',
    'french': 'French Fries', 'burger': 'Burger Rings'
}
```

In [86]:
```
df_transactions['BRAND'] = df_transactions['BRAND'].replace(brand_map)
```

In [87]:
```
print(f'BRAND: {df_transactions["BRAND"].unique()}')
print(f'BRAND: {df_transactions["BRAND"].nunique()}')
```

```
BRAND: ['Natural chip co.' 'CCS' 'Smiths' 'Kettle' 'Grain Waves' 'Doritos'
 'Twisties' 'Woolworths' 'Thins' 'Burger Rings' 'Cheezels' 'Infuzions'
 'Red Rock Deli' 'Pringles' 'Tyrrells' 'Cobs' 'French Fries' 'Tostitos'
 'Cheetos' 'Sunbites']
BRAND: 20
```

In [88]:
```
df_transactions.head()
```

Out[88]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES	transactions_per_day	BRAND	BRA
0	2018-10-17	1	1000	1	5	natural chip compny seasalt175g	2	6.0	NaN	Natural chip co.	Natu
1	2019-05-14	1	1307	348	66	ccs nacho cheese 175g	3	6.3	NaN	CCS	
2	2019-05-20	1	1343	383	61	smiths crinkle cut chips chicken 170g	2	2.9	NaN	Smiths	
3	2018-08-17	2	2373	974	69	smiths chip thinly s/cream&onion 175g	5	15.0	NaN	Smiths	
4	2018-08-18	2	2426	1038	108	kettle tortilla chpshny&jlpno chili 150g	3	13.8	NaN	Kettle	

In [93]:
```
#Creating Package size
df_transactions["PACK_SIZE"] = df_transactions["PROD_NAME"].str.extract(r'(\d+)').astype(float)
df_transactions.head()
```

Out[93]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES	transactions_per_day	BRAND	BRA
0	2018-10-17	1	1000	1	5	natural chip compny seasalt175g	2	6.0	NaN	Natural chip co.	Natu
1	2019-05-14	1	1307	348	66	ccs nacho cheese 175g	3	6.3	NaN	CCS	
2	2019-05-20	1	1343	383	61	smiths crinkle cut chips chicken 170g	2	2.9	NaN	Smiths	
3	2018-08-17	2	2373	974	69	smiths chip thinly s/cream&onion 175g	5	15.0	NaN	Smiths	
4	2018-08-18	2	2426	1038	108	kettle tortilla chpshny&jlpno chili 150g	3	13.8	NaN	Kettle	

In [94]:
```
df_transactions["PACK_SIZE"].describe()
```

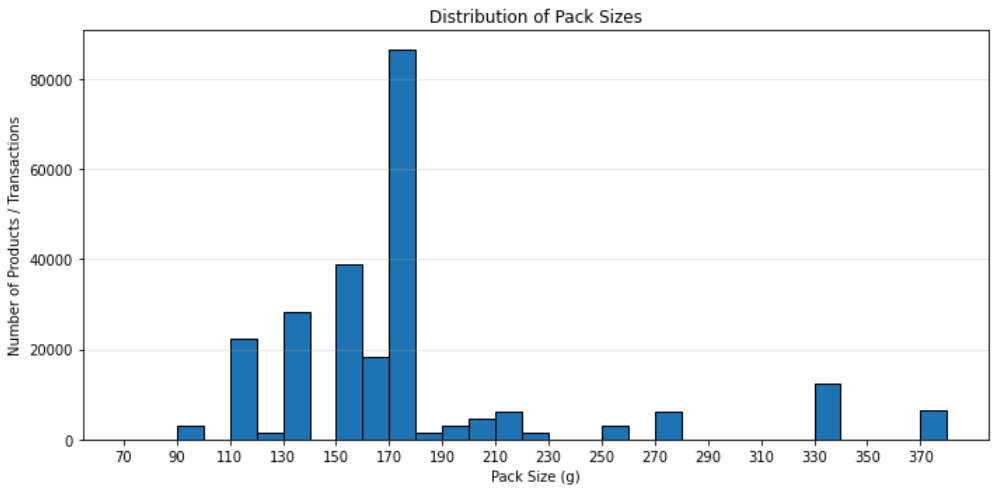
Out[94]:
```
count    243795.000000
mean        176.387079
std          59.172870
min          90.000000
25%         150.000000
50%         170.000000
75%         175.000000
max         380.000000
Name: PACK_SIZE, dtype: float64
```

In [95]:
```
# Plot histogram
plt.figure(figsize=(10,5))
plt.hist(df_transactions['PACK_SIZE'].dropna(), bins=range(70, 385, 10), edgecolor='black')  # bins of 10g

plt.title('Distribution of Pack Sizes')
```

```
plt.xlabel('Pack Size (g)')
plt.ylabel('Number of Products / Transactions')
plt.xticks(range(70, 385, 20))  # nicer tick marks
plt.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```



In [97]:
```
# Count how many transactions per pack size
pack_size_counts = df_transactions.groupby('PACK_SIZE').size().reset_index(name='transaction_count')

# Sort descending to see most common
pack_size_counts = pack_size_counts.sort_values(by='transaction_count', ascending=False)

print(pack_size_counts.head(10))  # top 10 pack sizes
```

```
    PACK_SIZE  transaction_count
9       175.0              66390
5       150.0              38765
3       134.0              25102
1       110.0              22387
8       170.0              19983
7       165.0              15297
17      330.0              12540
18      380.0               6416
16      270.0               6285
13      210.0               6272
```

Now it's time to merge both datasets

In [98]:
```
df_merged = pd.merge(
    df_transactions,
    df_purchase,
    on = "LYLTY_CARD_NBR",
    how = "left"
)
```

In [99]:
```
df_merged.head()
```

Out[99]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES	transactions_per_day	BRAND	BRA
0	2018-10-17	1	1000	1	5	natural chip compny seasalt175g	2	6.0	NaN	Natural chip co.	Natu
1	2019-05-14	1	1307	348	66	ccs nacho cheese 175g	3	6.3	NaN	CCS	
2	2019-05-20	1	1343	383	61	smiths crinkle cut chips chicken 170g	2	2.9	NaN	Smiths	
3	2018-08-17	2	2373	974	69	smiths chip thinly s/cream&onion 175g	5	15.0	NaN	Smiths	
4	2018-08-18	2	2426	1038	108	kettle tortilla chpshny&jlpno chili 150g	3	13.8	NaN	Kettle	

In [100…

```
df_merged.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 243795 entries, 0 to 243794
Data columns (total 15 columns):
 #   Column                Non-Null Count   Dtype
---  ------                --------------   -----
 0   DATE                  243795 non-null  datetime64[ns]
 1   STORE_NBR             243795 non-null  int64
 2   LYLTY_CARD_NBR        243795 non-null  int64
 3   TXN_ID                243795 non-null  int64
 4   PROD_NBR              243795 non-null  int64
 5   PROD_NAME             243795 non-null  object
 6   PROD_QTY              243795 non-null  int64
 7   TOT_SALES             243795 non-null  float64
 8   transactions_per_day  0 non-null       float64
 9   BRAND                 243795 non-null  object
 10  BRAND_CLEAN           243795 non-null  object
 11  Pack_size             243795 non-null  float64
 12  PACK_SIZE             243795 non-null  float64
 13  LIFESTAGE             243795 non-null  object
 14  PREMIUM_CUSTOMER      243795 non-null  object
dtypes: datetime64[ns](1), float64(4), int64(5), object(5)
memory usage: 27.9+ MB
```

In [101…

```
df_merged.isna().sum()
```

Out[101…

```
DATE                        0
STORE_NBR                   0
LYLTY_CARD_NBR              0
TXN_ID                      0
PROD_NBR                    0
PROD_NAME                   0
PROD_QTY                    0
TOT_SALES                   0
transactions_per_day    243795
BRAND                       0
BRAND_CLEAN                 0
Pack_size                   0
PACK_SIZE                   0
LIFESTAGE                   0
PREMIUM_CUSTOMER            0
dtype: int64
```

In [103…

```
print(os.getcwd())
```

```
/home/leda/Projects/Customer_segments
```

In [104…

```
df_merged.to_csv('/home/leda/Projects/Customer_segments/df_merged.csv', index=False)
```

Customers segment analysis

Metris on the dataset

In [126…

```
#Who spends the most on chips (total sales), describing customers by lifestage and
#how premium their general purchasing behaviour is
```

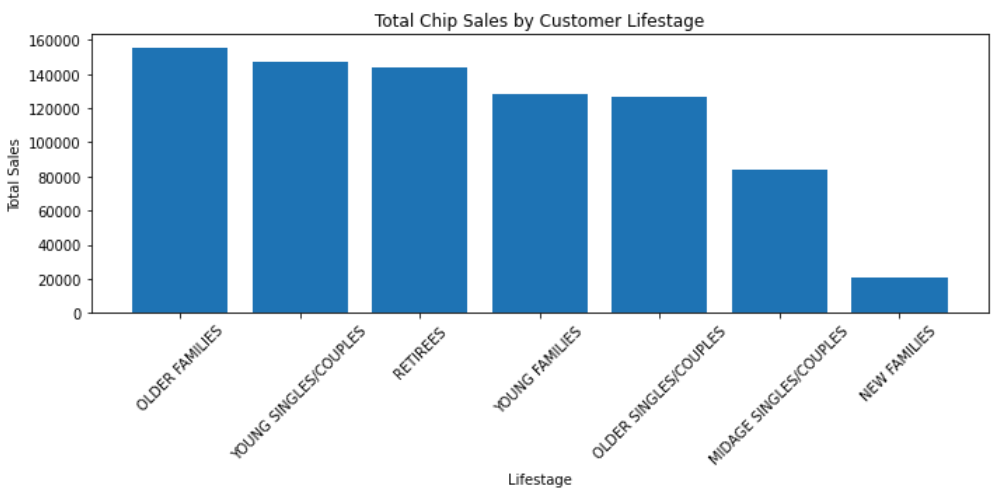
```
sales_by_lifestage = (
    df_merged
    .groupby(['LIFESTAGE','PREMIUM_CUSTOMER'],as_index = False)
    .agg(
        TOTAL_SALES = ('TOT_SALES','sum'),
        TOTAL_QTY = ('PROD_QTY','sum'))
    .sort_values(by='TOTAL_SALES', ascending=False)
)

print(sales_by_lifestage)
```

```
                LIFESTAGE PREMIUM_CUSTOMER  TOTAL_SALES  TOTAL_QTY
6           OLDER FAMILIES           Budget    155434.35      41282
19   YOUNG SINGLES/COUPLES       Mainstream    146980.20      35985
13                RETIREES       Mainstream    144079.95      37239
15          YOUNG FAMILIES           Budget    128575.15      34024
9    OLDER SINGLES/COUPLES           Budget    126742.20      32446
10   OLDER SINGLES/COUPLES       Mainstream    123643.10      32206
11   OLDER SINGLES/COUPLES          Premium    122749.15      31379
12                RETIREES           Budget    105302.90      26687
7           OLDER FAMILIES       Mainstream     95635.15      25491
14                RETIREES          Premium     90691.65      23023
16          YOUNG FAMILIES       Mainstream     85547.65      22878
1   MIDAGE SINGLES/COUPLES       Mainstream     84269.85      21027
17          YOUNG FAMILIES          Premium     77852.50      20614
8           OLDER FAMILIES          Premium     74522.60      19950
18   YOUNG SINGLES/COUPLES           Budget     56514.70      15257
2   MIDAGE SINGLES/COUPLES          Premium     53973.25      14210
20   YOUNG SINGLES/COUPLES          Premium     38661.90      10418
0   MIDAGE SINGLES/COUPLES           Budget     33028.30       8757
3             NEW FAMILIES           Budget     20474.65       5188
4             NEW FAMILIES       Mainstream     15899.50       4028
5             NEW FAMILIES          Premium     10701.00       2745
```

In [122…

```
plt.figure(figsize=(10,5))
plt.bar(sales_by_lifestage['LIFESTAGE'], sales_by_lifestage['TOTAL_SALES'])
plt.xticks(rotation=45)
plt.xlabel('Lifestage')
plt.ylabel('Total Sales')
plt.title('Total Chip Sales by Customer Lifestage')
plt.tight_layout()
plt.show()
```



- The highest spending segment is "Older Families" (Budget) due to high purchase volume rather than premium pricing.
- Mainstream customers, "Young Singles" and "Retirees" are major contributors to total chip sales due to frequent purchases.

In [128…

```
customer_counts = (
    df_merged
    .groupby(["LIFESTAGE", "PREMIUM_CUSTOMER"])["LYLTY_CARD_NBR"]
    .nunique()
    .reset_index(name="NUM_CUSTOMERS")
    .sort_values(by='NUM_CUSTOMERS', ascending=False)
)

print(customer_counts)
```

```
                 LIFESTAGE PREMIUM_CUSTOMER  NUM_CUSTOMERS
19   YOUNG SINGLES/COUPLES       Mainstream           7904
13                RETIREES       Mainstream           6326
10   OLDER SINGLES/COUPLES       Mainstream           4842
9    OLDER SINGLES/COUPLES           Budget           4828
11   OLDER SINGLES/COUPLES          Premium           4672
6           OLDER FAMILIES           Budget           4602
12                RETIREES           Budget           4371
15          YOUNG FAMILIES           Budget           3947
14                RETIREES          Premium           3802
18   YOUNG SINGLES/COUPLES           Budget           3617
1   MIDAGE SINGLES/COUPLES       Mainstream           3295
7           OLDER FAMILIES       Mainstream           2780
16          YOUNG FAMILIES       Mainstream           2680
20   YOUNG SINGLES/COUPLES          Premium           2464
17          YOUNG FAMILIES          Premium           2392
2   MIDAGE SINGLES/COUPLES          Premium           2357
8           OLDER FAMILIES          Premium           2227
0   MIDAGE SINGLES/COUPLES           Budget           1467
3             NEW FAMILIES           Budget           1086
4             NEW FAMILIES       Mainstream            827
5             NEW FAMILIES          Premium            574
```

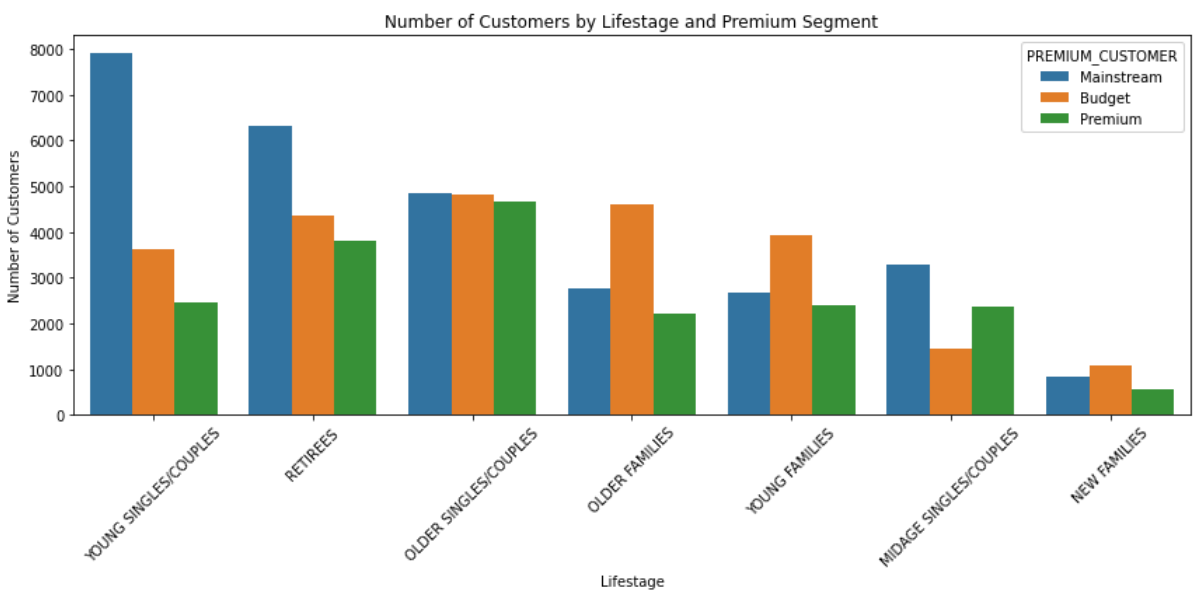
In [131…

```
plt.figure(figsize=(12,6))

sns.barplot(
    data=customer_counts,
    x="LIFESTAGE",
    y="NUM_CUSTOMERS",
    hue="PREMIUM_CUSTOMER"
)

plt.xticks(rotation=45)
plt.title("Number of Customers by Lifestage and Premium Segment")
plt.ylabel("Number of Customers")
plt.xlabel("Lifestage")

plt.tight_layout()
plt.show()
```



- "Older families" and "Young Families" are the ones that buy more chips per customer.

In [155…

```
#Average number of units per customer by Lifestage and Premiun_customer
avg_units = (
    df_merged
    .groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])
    .agg(
        TOTAL_QTY = ('PROD_QTY','sum'),
        NUM_CUSTOMERS = ('LYLTY_CARD_NBR','nunique')
    )
    .reset_index()
    .sort_values(by='TOTAL_QTY', ascending=False)
)
```

```
avg_units["AVG_UNITS_PER_CUSTOMER"] = avg_units["TOTAL_QTY"] / avg_units["NUM_CUSTOMERS"]
avg_units = avg_units.sort_values(by = "AVG_UNITS_PER_CUSTOMER", ascending = False)
print(avg_units)
```

```
                LIFESTAGE PREMIUM_CUSTOMER  TOTAL_QTY  NUM_CUSTOMERS  \
7          OLDER FAMILIES       Mainstream      25491           2780
6          OLDER FAMILIES           Budget      41282           4602
8          OLDER FAMILIES          Premium      19950           2227
15         YOUNG FAMILIES           Budget      34024           3947
17         YOUNG FAMILIES          Premium      20614           2392
16         YOUNG FAMILIES       Mainstream      22878           2680
9    OLDER SINGLES/COUPLES           Budget      32446           4828
11   OLDER SINGLES/COUPLES          Premium      31379           4672
10   OLDER SINGLES/COUPLES       Mainstream      32206           4842
1   MIDAGE SINGLES/COUPLES       Mainstream      21027           3295
12               RETIREES           Budget      26687           4371
14               RETIREES          Premium      23023           3802
2   MIDAGE SINGLES/COUPLES          Premium      14210           2357
0   MIDAGE SINGLES/COUPLES           Budget       8757           1467
13               RETIREES       Mainstream      37239           6326
4            NEW FAMILIES       Mainstream       4028            827
5            NEW FAMILIES          Premium       2745            574
3            NEW FAMILIES           Budget       5188           1086
19   YOUNG SINGLES/COUPLES       Mainstream      35985           7904
20   YOUNG SINGLES/COUPLES          Premium      10418           2464
18   YOUNG SINGLES/COUPLES           Budget      15257           3617

    AVG_UNITS_PER_CUSTOMER
7                 9.169424
6                 8.970448
8                 8.958240
15                8.620218
17                8.617893
16                8.536567
9                 6.720381
11                6.716396
10                6.651384
1                 6.381487
12                6.105468
14                6.055497
2                 6.028850
0                 5.969325
13                5.886658
4                 4.870617
5                 4.782230
3                 4.777164
19                4.552758
20                4.228084
18                4.218137
```

In [164…

```
#Average price per unit chips bought for each customer segments
avg_price_per_units = (
    df_merged
    .groupby(['LIFESTAGE','PREMIUM_CUSTOMER'])
    .agg(
        TOTAL_SALES = ('TOT_SALES','sum'),
        PRO_QTY = ('PROD_QTY','sum')
    )
    .reset_index()
)

avg_price_per_units["AVG_PRICE_PER_UNITS"] = avg_price_per_units["TOTAL_SALES"] / avg_price_per_units["PRO_QTY"]
avg_price_per_units = avg_price_per_units.sort_values(by = "AVG_PRICE_PER_UNITS", ascending = False)
print(avg_price_per_units)
```

```
                LIFESTAGE PREMIUM_CUSTOMER  TOTAL_SALES  PRO_QTY  \
19   YOUNG SINGLES/COUPLES       Mainstream    146980.20    35985
1   MIDAGE SINGLES/COUPLES       Mainstream     84269.85    21027
4             NEW FAMILIES       Mainstream     15899.50     4028
3             NEW FAMILIES           Budget     20474.65     5188
12                RETIREES           Budget    105302.90    26687
14                RETIREES          Premium     90691.65    23023
11   OLDER SINGLES/COUPLES          Premium    122749.15    31379
9    OLDER SINGLES/COUPLES           Budget    126742.20    32446
5             NEW FAMILIES          Premium     10701.00     2745
13                RETIREES       Mainstream    144079.95    37239
10   OLDER SINGLES/COUPLES       Mainstream    123643.10    32206
2   MIDAGE SINGLES/COUPLES          Premium     53973.25    14210
15          YOUNG FAMILIES           Budget    128575.15    34024
17          YOUNG FAMILIES          Premium     77852.50    20614
0   MIDAGE SINGLES/COUPLES           Budget     33028.30     8757
6           OLDER FAMILIES           Budget    155434.35    41282
7           OLDER FAMILIES       Mainstream     95635.15    25491
16          YOUNG FAMILIES       Mainstream     85547.65    22878
8           OLDER FAMILIES          Premium     74522.60    19950
20   YOUNG SINGLES/COUPLES          Premium     38661.90    10418
18   YOUNG SINGLES/COUPLES           Budget     56514.70    15257

    AVG_PRICE_PER_UNITS
19             4.084485
1              4.007697
4              3.947244
3              3.946540
12             3.945850
14             3.939176
11             3.911825
9              3.906250
5              3.898361
13             3.869061
10             3.839132
2              3.798258
15             3.778955
17             3.776681
0              3.771646
6              3.765185
7              3.751722
16             3.739298
8              3.735469
20             3.711067
18             3.704182
```

- "Young singles/couples" and "Midage singles/couples" pay more per packet of chips compared to their budget counterparts. This might be due to premium shoppers being more likely to buy healthy snacks and when they buy chips, they do it for entertainme/nt purposes rather than their own consumption.

In [166…

```python
#As the difference in average price per unit isn't large, I would be good to check
#if the difference is statistically different

#T-test between mainstream vs premium and budget midage, young singles and couples

# Function to perform t-test for a specific lifestage and two premium types
def ttest_avg_price(df, lifestage, group1, group2):
    # Filter data for the lifestage and customer groups
    data1 = df[(df['LIFESTAGE'] == lifestage) & (df['PREMIUM_CUSTOMER'] == group1)]
    data2 = df[(df['LIFESTAGE'] == lifestage) & (df['PREMIUM_CUSTOMER'] == group2)]

    # Calculate price per unit for each transaction
    price_per_unit1 = data1['TOT_SALES'] / data1['PROD_QTY']
    price_per_unit2 = data2['TOT_SALES'] / data2['PROD_QTY']

    # Perform Welch's t-test (does not assume equal variance)
    t_stat, p_value = ttest_ind(price_per_unit1, price_per_unit2, equal_var=False)

    print(f"Lifestage: {lifestage}, Groups: {group1} vs {group2}")
    print(f"t-statistic = {t_stat:.3f}, p-value = {p_value:.4f}\n")

# --- MIDAGE SINGLES/COUPLES ---
ttest_avg_price(df_merged, 'MIDAGE SINGLES/COUPLES', 'Mainstream', 'Premium')
ttest_avg_price(df_merged, 'MIDAGE SINGLES/COUPLES', 'Mainstream', 'Budget')

# --- YOUNG SINGLES/COUPLES ---
ttest_avg_price(df_merged, 'YOUNG SINGLES/COUPLES', 'Mainstream', 'Premium')
ttest_avg_price(df_merged, 'YOUNG SINGLES/COUPLES', 'Mainstream', 'Budget')
```

```
Lifestage: MIDAGE SINGLES/COUPLES, Groups: Mainstream vs Premium
t-statistic = 13.753, p-value = 0.0000

Lifestage: MIDAGE SINGLES/COUPLES, Groups: Mainstream vs Budget
t-statistic = 13.172, p-value = 0.0000

Lifestage: YOUNG SINGLES/COUPLES, Groups: Mainstream vs Premium
t-statistic = 24.204, p-value = 0.0000

Lifestage: YOUNG SINGLES/COUPLES, Groups: Mainstream vs Budget
t-statistic = 28.781, p-value = 0.0000
```

Insights on Youn Singles/Couples group

Brand Analysis

In [186…

```
#Focusing into an specific customer segments for insights

# Filter for Mainstream - YOUNG SINGLES/COUPLES
segment_df = df_merged[
    (df_merged['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') &
    (df_merged['PREMIUM_CUSTOMER'] == 'Mainstream')
]

other = df_merged[
    ~(
        (df_merged['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') &
        (df_merged['PREMIUM_CUSTOMER'] == 'Mainstream')
    )
]
```

In [187…

```
#Segment of interest
quantity_segment1 = segment_df['PROD_QTY'].sum()
quantity_other = other['PROD_QTY'].sum()

# Target segment
quantity_segment1_by_brand = (
    segment_df.groupby('BRAND')['PROD_QTY']
    .sum()
    .reset_index()
)

quantity_segment1_by_brand['targetSegment'] = (
    quantity_segment1_by_brand['PROD_QTY'] / quantity_segment1
)

# Other customers
quantity_other_by_brand = (
    other.groupby('BRAND')['PROD_QTY']
    .sum()
    .reset_index()
)

quantity_other_by_brand['other'] = (
    quantity_other_by_brand['PROD_QTY'] / quantity_other
)
```

In [188…

```
brand_proportions = pd.merge(
    quantity_segment1_by_brand[['BRAND', 'targetSegment']],
    quantity_other_by_brand[['BRAND', 'other']],
    on='BRAND'
)

brand_proportions['affinityToBrand'] = (
    brand_proportions['targetSegment'] / brand_proportions['other']
)
```

In [190…

```
brand_proportions = brand_proportions.sort_values(
    by='affinityToBrand', ascending=False
)

brand_proportions.head(21)
```

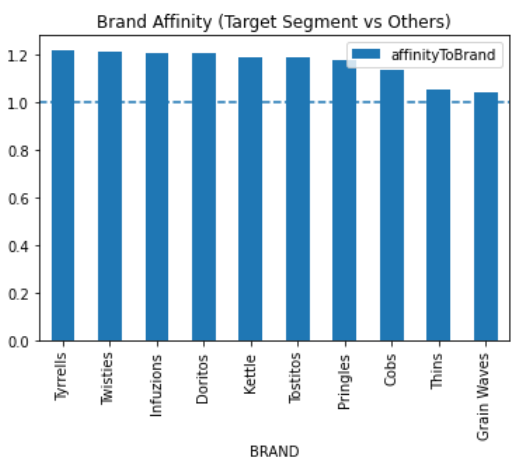
Out[190…

	BRAND	targetSegment	other	affinityToBrand
18	Tyrrells	0.031763	0.026011	1.221123
17	Twisties	0.046492	0.038347	1.212397
8	Infuzions	0.062054	0.051372	1.207918
5	Doritos	0.123579	0.102330	1.207658
9	Kettle	0.199305	0.167609	1.189108
16	Tostitos	0.045713	0.038449	1.188925
11	Pringles	0.120217	0.101884	1.179934
4	Cobs	0.044935	0.039534	1.136634
15	Thins	0.060775	0.057694	1.053409
7	Grain Waves	0.032930	0.031575	1.042919
3	Cheezels	0.018091	0.018878	0.958282
13	Smiths	0.093400	0.120115	0.777591
6	French Fries	0.003974	0.005830	0.681677
2	Cheetos	0.008087	0.012216	0.661954
12	Red Rock Deli	0.044102	0.068332	0.645406
10	Natural chip co.	0.019730	0.031237	0.631635
1	CCS	0.011255	0.019130	0.588318
14	Sunbites	0.006392	0.012736	0.501833
19	Woolworths	0.024260	0.050041	0.484805
0	Burger Rings	0.002946	0.006678	0.441078

In [191…

```
top_brands = brand_proportions.head(10)

top_brands.plot(
    x='BRAND',
    y='affinityToBrand',
    kind='bar'
)
plt.axhline(1, linestyle='--')  # baseline
plt.title("Brand Affinity (Target Segment vs Others)")
plt.show()
```



- The analysis of affinity shows that mainstream Young singles/couples are 22% more likely to purchase Tyrrells chips compared to the rest of the population.
- Also mainstream Youn singles/couples are 44% less likely to purchase Burger Rings.
- Brands like Doritos, Kettle, and Pringles should be targeted through brand-led promotions, in-store visibility, and digital marketing campaigns.

Package Size Analysiss

In [193…

```
# Target segment
pack_segment1 = (
    segment_df.groupby('PACK_SIZE')['PROD_QTY']
    .sum()
    .reset_index()
)

pack_segment1['targetSegment'] = pack_segment1['PROD_QTY'] / quantity_segment1

# Other customers
pack_other = (
    other.groupby('PACK_SIZE')['PROD_QTY']
    .sum()
    .reset_index()
)

pack_other['other'] = pack_other['PROD_QTY'] / quantity_other
```

In [ ]:

In [194…

```
pack_affinity = pd.merge(
    pack_segment1[['PACK_SIZE', 'targetSegment']],
    pack_other[['PACK_SIZE', 'other']],
    on='PACK_SIZE'
)

pack_affinity['affinityToPack'] = (
    pack_affinity['targetSegment'] / pack_affinity['other']
)
```

In [195…

```
pack_affinity = pack_affinity.sort_values(
    by='affinityToPack', ascending=False
)

pack_affinity.head(10)
```

Out[195…

	PACK_SIZE	targetSegment	other	affinityToPack
16	270.0	0.032041	0.025408	1.261087
18	380.0	0.032375	0.025902	1.249893
17	330.0	0.061692	0.050785	1.214781
3	134.0	0.120217	0.101884	1.179934
1	110.0	0.106989	0.090906	1.176918
13	210.0	0.029318	0.025433	1.152736
4	135.0	0.014867	0.013238	1.123098
15	250.0	0.014450	0.012939	1.116790
8	170.0	0.081312	0.081992	0.991708
5	150.0	0.155037	0.159434	0.972422

- Mainstream Young singles/couples are 26% more likely to purchase a 270g pack of chips compared to the rest of the population.

In [204…

```
segment_products_270 = segment_df[segment_df["PACK_SIZE"] == 270]['PROD_NAME'].unique()
print(segment_products_270)
```

```
['twisties cheese     270g' 'twisties chicken270g']
```

- Twisties cheese are the only brand that has 270g packs and so this may reflect a higher likelihood of purchasing Twisties

Conclusions

- Sales within the chips category are primarily driven by three key customer segments: Budget – Older Families, Mainstream – Young Singles/Couples, and Mainstream – Retirees. The higher total spend observed among Mainstream Young Singles/Couples and Retirees is largely attributable to their greater representation within the customer base rather than significantly higher individual spending. However, Mainstream customers, particularly mid-age and young singles/couples, also demonstrate a higher willingness to pay per packet, suggesting a tendency towards impulse-driven purchasing behaviour.

- Further analysis reveals that Mainstream Young Singles/Couples show a strong preference for certain brands, being approximately 22% more likely to purchase Tyrrells chips compared to the rest of the population. This indicates a clear opportunity to tailor product placement and promotional strategies towards this segment.

In [207…

```
from fpdf import FPDF

pdf = FPDF()
pdf.add_page()
pdf.set_font("Courier", size=10)

with open("Retail_strategy.ipynb", "r", encoding="utf-8") as f:
    for line in f:
        safe_line = line.encode('latin-1', 'replace').decode('latin-1')
        pdf.multi_cell(0, 5, safe_line.rstrip())

pdf.output("Retail_strategy.pdf")
```

Out[207…

```
''
```

In [ ]: